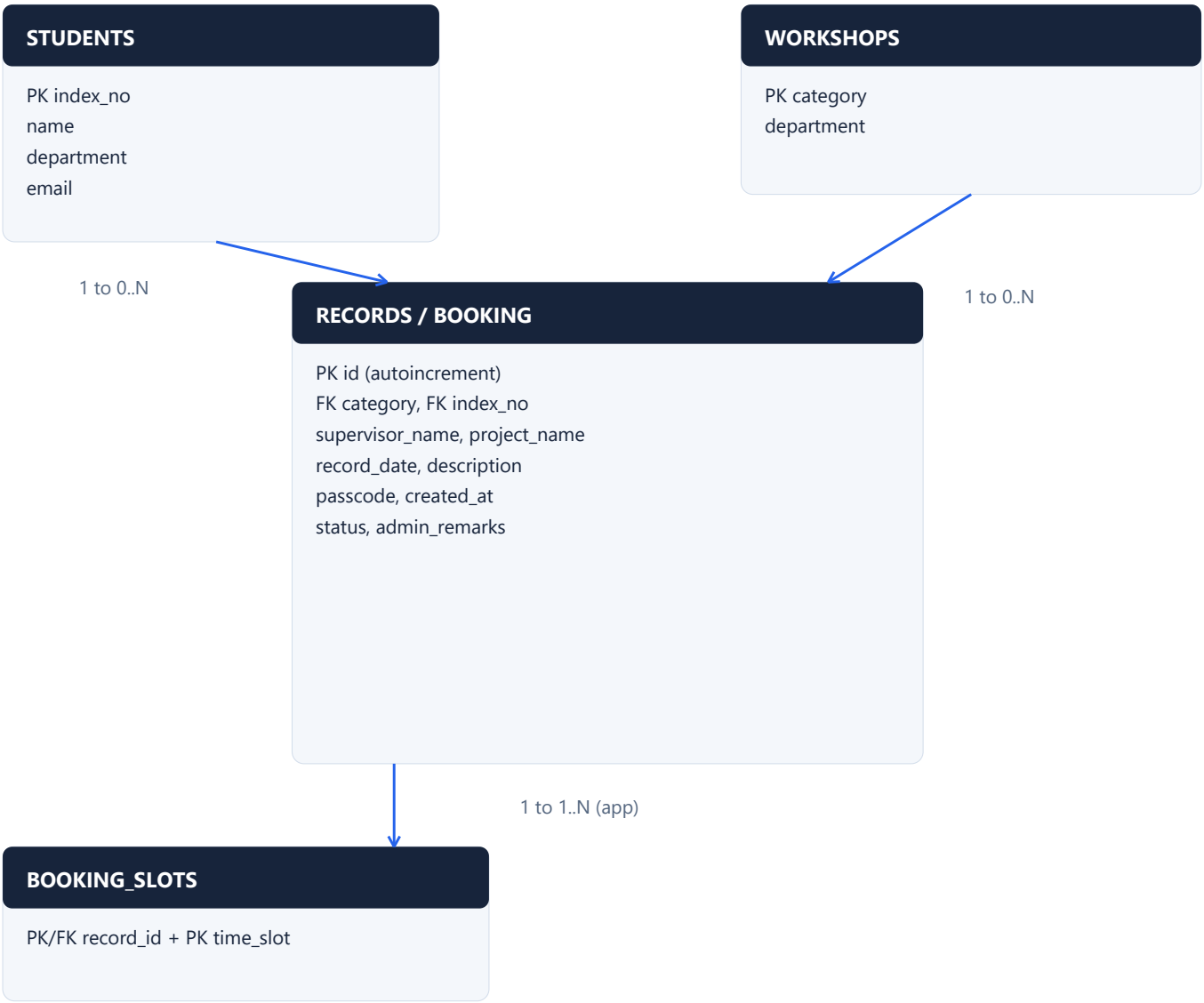
A3 / DESIGN AND MODELLING

Database design

SYNCLAB uses SQLite schema version 4. The four persisted relations separate workshop ownership, student profiles, allocation facts and selected time slots.



Cardinalities: a student/workshop may have zero or many records. Each valid application booking has one or more slot rows. SQL alone permits a record without a child slot; the application enforces the minimum.

Implementation basis: StudentWorkshopApp.pro and its compiled sources. Prepared 07 October 2026. All demonstration identities and bookings are fictional.

EXACT ACTIVE SQLITE NAMES

Relational schema and data dictionary

workshops

Attribute	Type	Key / constraint
category	TEXT	NOT NULL PRIMARY KEY
department	TEXT	NOT NULL

students

Attribute	Type	Key / constraint
index_no	TEXT	NOT NULL PRIMARY KEY
name	TEXT	NOT NULL
department	TEXT	NOT NULL
email	TEXT	NOT NULL

EXACT ACTIVE SQLITE NAMES

Relational schema and data dictionary

records

Attribute	Type	Key / constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
category	TEXT	NOT NULL REFERENCES workshops(category)
index_no	TEXT	NOT NULL REFERENCES students(index_no)
supervisor_name	TEXT	NOT NULL
project_name	TEXT	NOT NULL
record_date	TEXT	NOT NULL
description	TEXT	nullable
passcode	TEXT	NOT NULL
created_at	TEXT	NOT NULL
status	TEXT	NOT NULL DEFAULT 'Pending' CHECK(status IN ('Pending', 'Approve', 'Reject'))
admin_remarks	TEXT	nullable

booking_slots

Attribute	Type	Key / constraint
record_id	INTEGER	NOT NULL REFERENCES records(id) ON DELETE CASCADE
time_slot	TEXT	NOT NULL CHECK(length(trim(time_slot)) > 0 AND instr(time_slot, ',') = 0)

records_category_date indexes records(category, record_date). booking_slots has the composite primary key (record_id, time_slot).

DATABASE DECISIONS

Normalization and integrity

Atomic slots and separated facts

A selected slot is one booking_slots row. The composite key prevents the same slot appearing twice in the same booking. Student facts depend on index_no; workshop ownership depends on category; allocation facts depend on id. The database stores the student and workshop departments separately because they describe different facts.

Normal form	Decision
1NF	Store one atomic slot per child row; no repeating slot columns.
2NF	Non-key facts depend on their whole key; booking_slots has no non-key attributes.
3NF	Separate student identity and workshop ownership from records.
BCNF / 4NF	Compatible under the stated key dependencies and the absence of additional independent multivalued facts. Slots are a separate relation.

Integrity at two layers

SQLite enforces primary and foreign keys, legal status values and non-empty comma-free slot text. The application validates required inputs, email/date format, at least one slot, key collisions and cross-record occupancy. add/update/status operations use BEGIN IMMEDIATE and commit or roll back together.

Limits of the schema

passcode has no SQL UNIQUE constraint. A slot conflict across different records is prevented by application transaction logic, not a cross-record unique index. Changing a student profile updates all joined views for the same index. Raw external SQL writers must follow the same business rules.

REPRODUCIBLE DATABASE ARTIFACTS

SQL creation and sample data

Run create_tables.sql first in a new empty SQLite database, then sample_data.sql. The application creates its schema automatically for normal first use. The supplied SQL provides a separate, reproducible demonstration dataset.

```
-- SYNCLAB active schema v4, transcribed from database/databasemanager.cpp.
-- The caller must enable foreign keys for every connection.
PRAGMA foreign_keys = ON;
BEGIN IMMEDIATE;
CREATE TABLE workshops (
    category TEXT NOT NULL PRIMARY KEY,
    department TEXT NOT NULL
);
CREATE TABLE students (
    index_no TEXT NOT NULL PRIMARY KEY,
    name TEXT NOT NULL,
    department TEXT NOT NULL,
    email TEXT NOT NULL
);
CREATE TABLE records (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    category TEXT NOT NULL REFERENCES workshops(category),
    index_no TEXT NOT NULL REFERENCES students(index_no),
    supervisor_name TEXT NOT NULL,
    project_name TEXT NOT NULL,
    record_date TEXT NOT NULL,
    description TEXT,
    passcode TEXT NOT NULL,
    created_at TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'Pending'
        CHECK(status IN ('Pending', 'Approve', 'Reject')),
    admin_remarks TEXT
);
CREATE TABLE booking_slots (
    record_id INTEGER NOT NULL REFERENCES records(id) ON DELETE CASCADE,
    time_slot TEXT NOT NULL
        CHECK(length(trim(time_slot)) > 0 AND instr(time_slot, ',') = 0),
    PRIMARY KEY(record_id, time_slot)
);
CREATE INDEX records_category_date ON records(category, record_date);
PRAGMA user_version = 4;
COMMIT;
```

Sample dataset

sample_data.sql seeds ten workshops, five fictional students, eight allocations and eleven selected slots. Dates are 20-24 October 2026. It includes Pending, Approve and Reject records to demonstrate joins, status workload and released slots.

Sample data is for assessment and testing only. It is not an import of real student information.

PERSISTENCE BEHAVIOR

Migration and database verification

Version 4 migration

createTable() detects the legacy schema and, for a disk-backed migration, creates a separate .pre-4nf.bak copy using VACUUM INTO. It transforms legacy fields through staging tables inside a transaction, preserves record identifiers, normalizes slot memberships, checks foreign keys and writes user_version = 4. A newer unsupported version is refused.

Executed SQL verification

The exact supplied schema and demo data were executed with Python sqlite3. Twenty checks passed, including integrity_check, foreign_key_check, required fields, status validation, composite-slot uniqueness and deletion cascade. These SQL checks are separate from the compiled Qt integration tests.

Evidence	Location
CREATE statements	section A3 / SQL/create_tables.sql
Small INSERT dataset	section A3 / SQL/sample_data.sql
Five SELECT scripts and exported tables	section C8
Demo database	section C8 / synclab_demonstration.db
SQL checks and metadata	verification_results.json and schema_metadata.json

Recovery scope

The migration backup is automatic only when a legacy migration is detected. There is no user-facing backup/restore command. Routine recovery uses a copy of the selected database while SYNCLAB is closed. QSettings configuration is stored separately.